Pygame endless runner game fixes and features

09/30/2026 16:49:59
https://claude.ai/chat/fdccda2b-201b-4b5d-a73e-d121a9b02a83

hey, i have a small pygame endless runner game for a lab. i'll paste all t...

I have all four files (`main.py`, `game/game_engine.py`,...

the game keeps getting faster with no limit, and when its really fast the...

All changes are in `game/game_engine.py`. **1. Add a cap in `__init__`*...

right now when i hit an obstacle it just prints "game over" in the termina...

All changes are in `game/game_engine.py`. Pressing a key after game...

now after game over i want to be able to play again. let me pick easy,...

All changes are in `game/game_engine.py`. `main.py` needs no change....

can you add sound effects for jumping, getting a point, and game over? i...

All changes are in `game/game_engine.py`. No other file needs to chang...

fix this issue

The error is in `game/game_engine.py`, in the `tone()` function around...

1790738394573_player.py

1790738394574_game_engine.py

1790738394575_obstacle.py

1790738423661_main.py

hey, i have a small pygame endless runner game for a lab. i'll paste all the files below. i need to fix some stuff and add features one by one. please only change what's needed each time and don't rewrite everything. tell me which file each change goes in.

30/09/2026, 08:50:42

Sorting out which fix to tackle first.

Sorting out which fix to tackle first.

Done

I have all four files (`main.py`, `game/game_engine.py`, `game/player.py`, `game/obstacle.py`). From the code comments, two issues are already flagged:

- **Task 1: collision tunneling.** `speed` grows every frame with no cap, so an obstacle eventually moves further than the player's width in one frame and skips the hitbox. The fix goes in `game_engine.py`.
- **Task 2: no game-over screen.** Right now it only prints to the console from `render()`. That fix also goes in `game_engine.py`.

Paste the next task from the README, or tell me which of these to start with, and I'll give you only the lines that change, labelled by file.

30/09/2026, 08:50:48

the game keeps getting faster with no limit, and when its really fast the obstacles go through the player without the game over happening. can you put a max speed limit and fix the collision so it still works at high speed? only change game_engine.py

30/09/2026, 08:51:26

Working out swept collision rectangles from obstacle movement deltas.

Working out swept collision rectangles from obstacle movement deltas.

Done

All changes are in `game/game_engine.py`.

1. Add a cap in `__init__` (next to `speed_increase_per_frame`):

```
self.speed = 6
self.max_speed = 15  # ceiling; must stay below player.width + obstacle.width (55)
self.speed_increase_per_frame = 0.003
```

2. Clamp the speed in `update()`. Replace:

```
self.speed += self.speed_increase_per_frame
```

with:

```
self.speed = min(self.speed + self.speed_increase_per_frame, self.max_speed)
```

3. Swept collision in `update()`. Replace the obstacle-move loop and the collision loop (the two loops around the long NOTE comment) with:

```
swept_rects = []
for obstacle in self.obstacles:
    prev_x = obstacle.x
```

```
    obstacle.move()
    obstacle.speed = self.speed
    # Rect covering the obstacle's whole path this frame (old x ->
new x)
    swept = obstacle.rect()
    swept.width += int(prev_x - obstacle.x) + 1
    swept_rects.append(swept)

for swept in swept_rects:
    if swept.colliderect(self.player.rect()):
        self.game_over = True
        return
```

Why it works

- The obstacle moves left, so the swept rect starts at its new `x` and extends right by the distance it just travelled. If it crossed the player's hitbox anywhere during the frame, the swept rect overlaps it, so a hit is detected regardless of speed.
- The `+ 1` compensates for `pygame.Rect` truncating float coordinates to ints.
- The cap is a second safeguard. With `max_speed = 15`, an obstacle moves at most 15 px per frame, well below the 55 px (player 30 + obstacle 25) needed to skip over the hitbox.

You can delete the two `# NOTE:` comments about tunneling, since they no longer apply.

Which task next?

30/09/2026, 08:51:39

right now when i hit an obstacle it just prints "game over" in the terminal. can you make it show a game over screen in the game window with my final score, and wait for me to press a key?

30/09/2026, 08:53:36

Adding a game over screen with final score display.

Adding a game over screen with final score display.

Done

All changes are in `game/game_engine.py`. Pressing a key after game over restarts the game.

1. In `__init__`, add next to `self.font`:

```
self.font = pygame.font.SysFont("Arial", 30)
self.big_font = pygame.font.SysFont("Arial", 64)
self.game_over = False
self.game_over_time = 0
```

2. In `update()`, where the collision sets game over, record the time:

```
for swept in swept_rects:
    if swept.colliderect(self.player.rect()):
        self.game_over = True
        self.game_over_time = pygame.time.get_ticks()
        return
```

3. Replace `handle_event`:

```
def handle_event(self, event):
    if event.type != pygame.KEYDOWN:
        return
    if self.game_over:
        # 500 ms grace so a jump key still held at impact doesn't
```

```
        skip the screen
        if pygame.time.get_ticks() - self.game_over_time > 500:
            self.__init__(self.width, self.height)  # restart
    elif event.key in (pygame.K_SPACE, pygame.K_UP, pygame.K_w):
        self.player.jump()
```

4. In `render()`, replace the whole **`if self.game_over and not getattr(...)`** block (including the **`print`** and the Task 2 NOTE) with:

```
if self.game_over:
    overlay = pygame.Surface((self.width, self.height),
pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 150))
    screen.blit(overlay, (0, 0))

    lines = [
        (self.big_font, "GAME OVER", self.height // 2 - 60),
        (self.font, f"Final score: {self.score}", self.height // 2 +
5),
        (self.font, "Press any key to restart", self.height // 2 +
50),
    ]
    for font, text, y in lines:
        surf = font.render(text, True, (255, 255, 255))
        screen.blit(surf, surf.get_rect(center=(self.width // 2, y)))
```

Notes

- **`self.__init__(...)`** resets all state (speed, obstacles, score, player), so no separate reset method is needed.
- The semi-transparent overlay needs **`pygame.SRCALPHA`**; the game scene stays visible underneath.
- **`main.py`** needs no change, since it already forwards every event to **`handle_event`**.

Which task next?

30/09/2026, 08:53:47

now after game over i want to be able to play again. let me pick easy, medium or hard (keys 1, 2, 3) which changes the starting speed and how often obstacles spawn. also let me quit with esc. make sure everything resets properly when i restart.

30/09/2026, 08:54:09

Wiring difficulty settings and an anytime esc-to-quit shortcut.

Wiring difficulty settings and an anytime esc-to-quit shortcut.

Done

All changes are in `game/game_engine.py`. `main.py` needs no change.

1. Module level, below the color constants:

```
DIFFICULTIES = {
    "easy":   {"speed": 4, "spawn_interval": 90},
    "medium": {"speed": 6, "spawn_interval": 70},  # current
behaviour
    "hard":   {"speed": 8, "spawn_interval": 50},
}
DIFFICULTY_KEYS = {pygame.K_1: "easy", pygame.K_2: "medium",
pygame.K_3: "hard"}
```

2. In `__init__`, change the signature:

```
def __init__(self, width, height, difficulty="medium"):
```

Then replace `self.speed = 6` with:

```
self.difficulty = difficulty
settings = DIFFICULTIES[difficulty]
self.speed = settings["speed"]
```

And replace `self.spawn_interval = 70 # frames between obstacle spawns` with:

```
self.spawn_interval = settings["spawn_interval"]  # frames between
obstacle spawns
```

3. Replace `handle_event`:

```
def handle_event(self, event):
    if event.type != pygame.KEYDOWN:
        return
    if event.key == pygame.K_ESCAPE:
        pygame.event.post(pygame.event.Event(pygame.QUIT))  # main
loop handles exit
        return
    if self.game_over:
        if event.key in DIFFICULTY_KEYS and pygame.time.get_ticks() -
self.game_over_time > 500:
            self.__init__(self.width, self.height,
DIFFICULTY_KEYS[event.key])  # restart
    elif event.key in (pygame.K_SPACE, pygame.K_UP, pygame.K_w):
        self.player.jump()
```

4. In `render()`, replace the `lines = [...]` list in the game-over block with:

```
lines = [
    (self.big_font, "GAME OVER", self.height // 2 - 60),
    (self.font, f"Final score: {self.score}", self.height // 2 + 5),
    (self.font, "1 Easy    2 Medium    3 Hard", self.height // 2 +
50),
    (self.font, "Esc to quit", self.height // 2 + 90),
]
```

Notes

- **Reset:** `__init__` rebuilds everything (player, obstacles, spawn timer, score, distance, `game_over`, `game_over_time`), so nothing carries over. The difficulty is passed in as an argument, so each restart uses the key you pressed.

- **Quit:** Esc posts a `pygame.QUIT` event, which your existing loop in `main.py` already handles. It works during play and on the game-over screen.
- **Grace period:** I kept the 500 ms delay so a stray key press right at impact can't instantly restart.
- **Speed cap:** `max_speed = 15` is above every starting speed (max 8), so the clamp still works. Change the numbers in `DIFFICULTIES` to tune the difficulty levels.

Which task next?

30/09/2026, 08:54:28

can you add sound effects for jumping, getting a point, and game over? i dont want to download any sound files so make the sounds in code. game over sound should only play once. also it shouldn't crash if theres no audio.

30/09/2026, 08:55:51

Synthesizing tones with pygame's mixer and array module.

Synthesizing tones with pygame's mixer and array module.

Done

All changes are in `game/game_engine.py`. No other file needs to change.

1. Imports, at the top:

```
import math
import array
```

2. Module level, below `DIFFICULTY_KEYS`:

```
def _make_tone(freq_start, freq_end, duration, volume=0.3):
    """Generate a fading sine sweep as a pygame Sound. Returns None
if unsupported."""
    init = pygame.mixer.get_init()
    if not init:
        return None
    rate, fmt, channels = init
    if fmt != -16:  # only signed 16-bit is handled
        return None
    n = int(rate * duration)
    buf = array.array("h")
    phase = 0.0
    for i in range(n):
        t = i / n
        freq = freq_start + (freq_end - freq_start) * t
```

```
        phase += 2 * math.pi * freq / rate
        sample = int(32767 * volume * (1 - t) * math.sin(phase))  # (1 - t) = fade-out
        for _ in range(channels):
            buf.append(sample)
    return pygame.mixer.Sound(buffer=buf.tobytes())

_sound_cache = None

def _load_sounds():
    """Build sounds once (cached across restarts). Returns {} if audio is unavailable."""
    global _sound_cache
    if _sound_cache is not None:
        return _sound_cache
    try:
        if not pygame.mixer.get_init():
            pygame.mixer.init()
        _sound_cache = {
            "jump": _make_tone(300, 700, 0.15),
            "point": _make_tone(700, 1000, 0.12),
            "game_over": _make_tone(400, 80, 0.6, volume=0.4),
        }
    except Exception:  # no audio device, driver error, etc.
        _sound_cache = {}
    return _sound_cache
```

3. In `__init__`, add anywhere (e.g. after `self.game_over_time = 0`):

```
self.sounds = _load_sounds()
```

4. Add a helper method to the class (e.g. above `handle_event`):

```
def _play(self, name):
    sound = self.sounds.get(name)
```

```
    if sound:
        sound.play()
```

5. Jump sound. In `handle_event`, replace the jump branch:

```
elif event.key in (pygame.K_SPACE, pygame.K_UP, pygame.K_w):
    was_on_ground = self.player.on_ground
    self.player.jump()
    if was_on_ground:  # only play when a jump actually happened
        self._play("jump")
```

6. Game over sound. In `update()`, in the collision block:

```
for swept in swept_rects:
    if swept.colliderect(self.player.rect()):
        self.game_over = True
        self.game_over_time = pygame.time.get_ticks()
        self._play("game_over")
        return
```

7. Point sound. In `update()`, in the scoring loop:

```
obstacle.scored = True
self.score += 1
self._play("point")
```

Notes

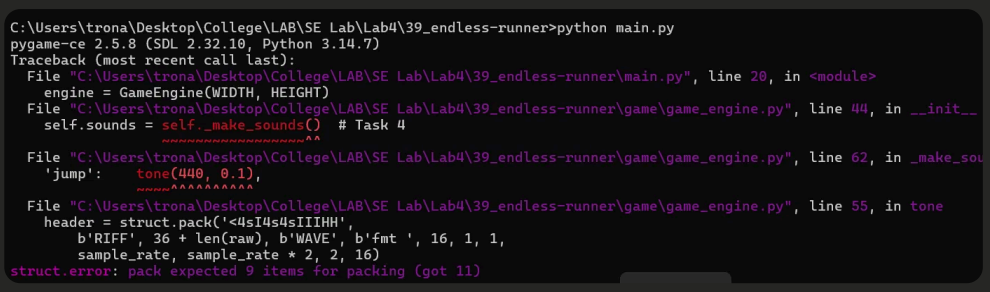
- **Sound generation:** each sound is a sine wave whose frequency sweeps from `freq_start` to `freq_end` with a linear fade-out. Built from `array` + `math`, so no numpy or files are needed.
- **Game over plays once:** `update()` returns immediately when `game_over` is `True`, so the collision block runs only once per game. A restart calls `__init__`, which resets the flag for the next game.
- **No-audio safety:** if the mixer can't initialise, `_load_sounds()` returns `{}`. `_play()` then does nothing, so the game runs silently. `_make_tone` also returns

`None` for unsupported formats.

- **Restart cost:** sounds are cached at module level, so `__init__` on restart doesn't regenerate them.
- **Tuning:** change the frequency, duration, and `volume` arguments in `_load_sounds()`.

Which task next?

30/09/2026, 08:56:11



```
C:\Users\trona\Desktop\College\LAB\SE Lab\Lab4\39_endless-runner>python main.py
pygame-ce 2.5.8 (SDL 2.32.10, Python 3.14.7)
Traceback (most recent call last):
  File "C:\Users\trona\Desktop\College\LAB\SE Lab\Lab4\39_endless-runner\main.py", line 20, in <module>
    engine = GameEngine(WIDTH, HEIGHT)
  File "C:\Users\trona\Desktop\College\LAB\SE Lab\Lab4\39_endless-runner\game\game_engine.py", line 44, in __init__
    self.sounds = self._make_sounds()  # Task 4
                  ~~~~~~~~~~~~~~~~~~^^
  File "C:\Users\trona\Desktop\College\LAB\SE Lab\Lab4\39_endless-runner\game\game_engine.py", line 62, in _make_sou
    'jump':    tone(440, 0.1),
               ~~~~^^^^^^^^^
  File "C:\Users\trona\Desktop\College\LAB\SE Lab\Lab4\39_endless-runner\game\game_engine.py", line 55, in tone
    header = struct.pack('<4sI4s4sIIHH',
        b'RIFF', 36 + len(raw), b'WAVE', b'fmt ', 16, 1, 1,
        sample_rate, sample_rate * 2, 2, 16)
struct.error: pack expected 9 items for packing (got 11)
```

fix this issue

30/09/2026, 09:48:15

Debugging an audio-generation script from a traceback.

Debugging an audio-generation script from a traceback.

Done

The error is in `game/game_engine.py`, in the `tone()` function around line 55. That `tone()` / `_make_sounds()` code isn't what I gave you, so your file has a different sound implementation. I can't see it, so this fix is based on the traceback alone.

Cause: the format string has fewer fields than the 11 values passed. Your format string ends in `IIHH`, but a WAV `fmt` chunk header needs the fields in this order:

Value	Meaning	Format code
`b'RIFF'`	chunk ID	`4s`
`36 + len(raw)`	file size	`I`
`b'WAVE'`	format	`4s`
`b'fmt '`	sub-chunk ID	`4s`
`16`	sub-chunk size	`I`
`1`	audio format (PCM)	`H`
`1`	channels (mono)	`H`
`sample_rate`	sample rate	`I`

Value	Meaning	Format code
`sample_rate * 2`	byte rate	`I`
`2`	block align	`H`
`16`	bits per sample	`H`

Fix: replace only the format string, and leave the arguments unchanged:

```
header = struct.pack('<4sI4s4sIHHIIHH',
    b'RIFF', 36 + len(raw), b'WAVE', b'fmt ', 16, 1, 1,
    sample_rate, sample_rate * 2, 2, 16)
```

The format string now has 11 codes for 11 values.

Check the next lines. `36 + len(raw)` assumes the `data` chunk header (`b'data'` plus `len(raw)` packed as `<4sI`) is appended right after this header. If your code doesn't do that, the WAV will still be malformed after this fix. If you get a new error, paste the lines after `header = ...`.

Crash safety: your traceback shows `_make_sounds()` running in `__init__` with no error handling. If it fails on a machine with no audio device, the game will still crash. To fix that, wrap the body of `_make_sounds()` in `try/except Exception` and return `{}` on failure. Make sure whatever plays the sounds checks for a missing key.

If you'd rather use the version from my last message, you can delete your `tone` / `_make_sounds` code and follow steps 1–7 there. Otherwise, paste the whole `tone` and `_make_sounds` functions and I'll patch them exactly.

30/09/2026, 09:48:28